스테이킹 서비스

소개

스테이킹 서비스는 Oasis Core 네트워크에서 Proof of Stake (PoS) 합의 메커니즘을 구현하는 핵심 서비스입니다. 이 서비스는 토큰 관리, 위임(delegation), 밸리데이터 선정, 보상 분배, 슬래싱(slashing) 등의 기능을 제공합니다.

주요 특징

- **Proof of Stake 메커니즘**: 스테이킹된 토큰을 기반으로 한 합의 참여
- **위임 시스템**: 토큰 보유자가 밸리데이터에게 토큰을 위임
- **자동 보상 분배**: 에포크별 블록 생성 및 검증 보상 자동 분배
- **슬래싱 메커니즘**: 악의적 행동에 대한 자동 처벌 시스템
- **유연한 수수료 구조**: 밸리데이터별 수수료 스케줄 설정

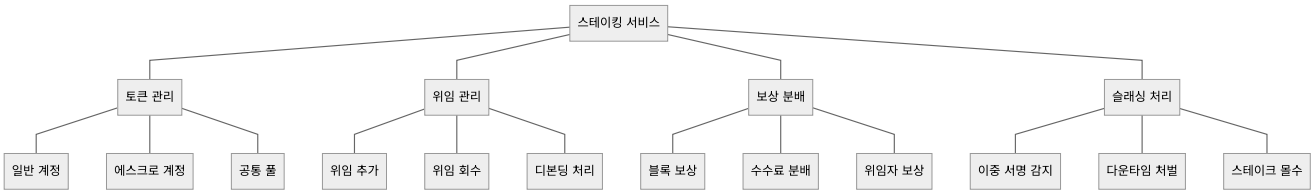
관련 프로세스

- **합의 프로세스**: 밸리데이터 선정과 투표권 계산
- **비콘 서비스**: 에포크 기반 보상 분배와 위임 처리
- **스케줄러 서비스**: 스테이킹 기반 위원회 구성
- **거버넌스 서비스**: 스테이킹 파라미터 변경 제안

아키텍처

전체 구조

스테이킹 서비스는 토큰 관리, 위임 관리, 보상 분배, 슬래싱 처리의 네 가지 주요 영역으로 구성됩니다.



주요 컴포넌트

계정 관리자

역할: 다양한 유형의 계정과 잔액 관리

주요 기능:

- 일반 계정 잔액 관리
- 에스크로 계정 관리
- 공통 풀 관리
- 거버넌스 예치금 관리

위임 관리자

역할: 토큰 위임과 회수 처리

주요 기능:

- 위임 추가 및 제거
- 디본딩 기간 관리
- 위임 정보 추적

- 수수료 계산

보상 분배기

역할: 블록 생성 및 검증 보상 분배

주요 기능:

- 에포크별 보상 계산
- 밸리데이터 수수료 적용
- 위임자 보상 분배
- 공통 풀 기여금 처리

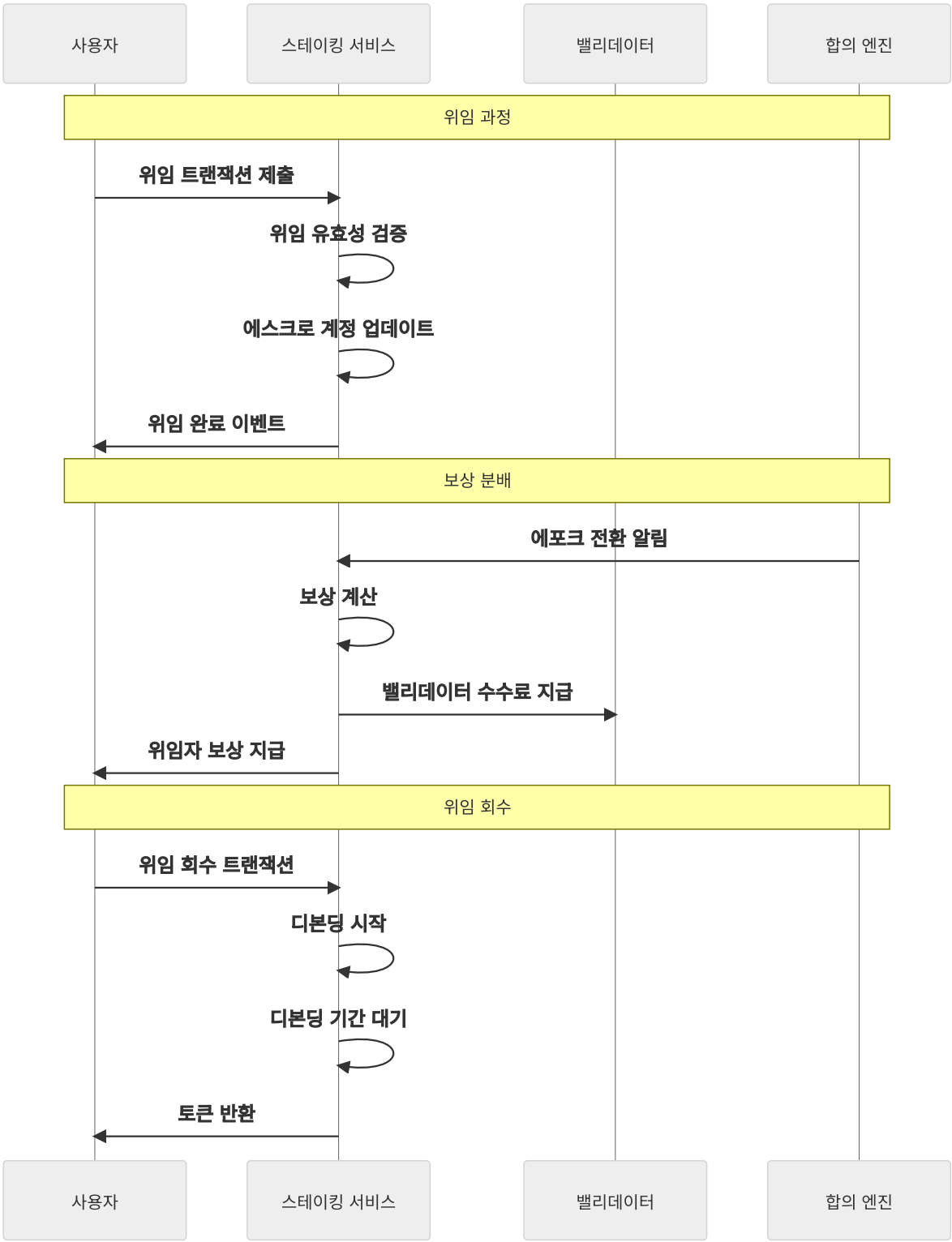
슬래싱 처리기

역할: 악의적 행동에 대한 처벌 실행

주요 기능:

- 이중 서명 감지 및 처벌
- 다운타임 추적 및 처벌
- 스테이크 몰수 실행
- 슬래싱 이벤트 기록

스테이킹 과정


사용자
스테이킹 서비스
밸리데이터
합의 엔진
위임 과정
위임 트랜잭션 제출
위임 유효성 검증
에스크로 계정 업데이트
위임 완료 이벤트
보상 분배
에포크 전환 알림
보상 계산
밸리데이터 수수료 지급
위임자 보상 지급
위임 회수
위임 회수 트랜잭션
디본딩 시작
디본딩 기간 대기
토큰 반환
사용자
스테이킹 서비스
밸리데이터
합의 엔진

PoS 메커니즘 설명

Proof of Stake 개요

Oasis Core의 PoS 시스템은 토큰 보유량에 비례하여 네트워크 참여 권한을 부여하는 합의 메커니즘입니다.

밸리데이터 선정 과정

1. **스테이킹 요구사항**: 최소 스테이킹 임계값 충족
2. **등록 과정**: 레지스트리에 밸리데이터 노드 등록
3. **투표권 계산**: 총 스테이킹 양에 비례한 투표권 할당
4. **활성 세트 선정**: 상위 스테이킹 밸리데이터들로 활성 세트 구성

```
// 밸리데이터 투표권 계산
func calculateVotingPower(totalStake *quantity.Quantity, maxVotingPower int64)
// 투표권은 스테이킹 양에 비례하되 최대값으로 제한
if totalStake.IsZero() {
return 0
}
// 스테이킹 양을 투표권으로 변환 (정규화)
votingPower := totalStake.ToBigInt().Int64()
if votingPower > maxVotingPower {
votingPower = maxVotingPower
}
return votingPower
```

```
}
```

위임 시스템

위임 시스템을 통해 토큰 보유자는 직접 밸리데이터를 운영하지 않고도 네트워크에 참여할 수 있습니다.

위임 과정

1. **위임 트랜잭션**: 사용자가 특정 밸리데이터에게 토큰 위임
2. **에스크로 계정**: 위임된 토큰은 에스크로 계정으로 이동
3. **쉐어 발행**: 위임 비율에 따른 쉐어 발행
4. **투표권 증가**: 밸리데이터의 총 투표권 증가

```
// 위임 처리 구조체
type Escrow struct {
Account Address `json:"account"`  // 밸리데이터 주소
Amount quantity.Quantity `json:"amount"`  // 위임할 토큰 양
}


// 위임 처리 함수
func (app *stakingApplication) addEscrow(
ctx *api.Context,
state *stakingState.MutableState,
from Address,
escrow *Escrow,
```

```go
) error {
// 위임자 계정에서 토큰 차감
fromAccount, err := state.Account(from)
if err != nil {
return err
}
if fromAccount.General.Balance.Cmp(&escrow.Amount) < 0 {
return api.ErrInsufficientBalance
}
// 밸리데이터 에스크로 계정으로 토큰 이동
escrowAccount, err := state.Account(escrow.Account)
if err != nil {
return err
}
// 쉐어 계산 및 발행
shares, err := escrowAccount.Escrow.Active.Deposit(
&fromAccount.Escrow.Active.Balance,
&fromAccount.General.Balance,
&escrow.Amount,
)
if err != nil {
return err
}
```

```
// 계정 상태 업데이트
state.SetAccount(from, fromAccount)
state.SetAccount(escrow.Account, escrowAccount)
return nil
}
```

스테이킹, 언스테이킹, 슬래싱 프로세스

스테이킹 프로세스

1. 직접 스테이킹

밸리데이터가 자신의 토큰을 스테이킹하는 과정:

```
// 자체 스테이킹 예시
func selfStake(validator Address, amount *quantity.Quantity) *transaction.Trans
escrow := &api.Escrow{
Account: validator, // 자신에게 위임
Amount: *amount,
}
return api.NewAddEscrowTx(0, nil, escrow)
}
```

2. 위임 스테이킹

일반 사용자가 밸리데이터에게 토큰을 위임하는 과정:

```
// 위임 스테이킹 예시
func delegateStake(delegator, validator Address, amount *quantity.Quantity) *tr
escrow := &api.Escrow{
Account: validator, // 밸리데이터에게 위임
Amount: *amount,
}
return api.NewAddEscrowTx(0, nil, escrow)
}
```

언스테이킹 (위임 회수) 프로세스

언스테이킹은 디본딩 기간을 거쳐 단계적으로 진행됩니다.

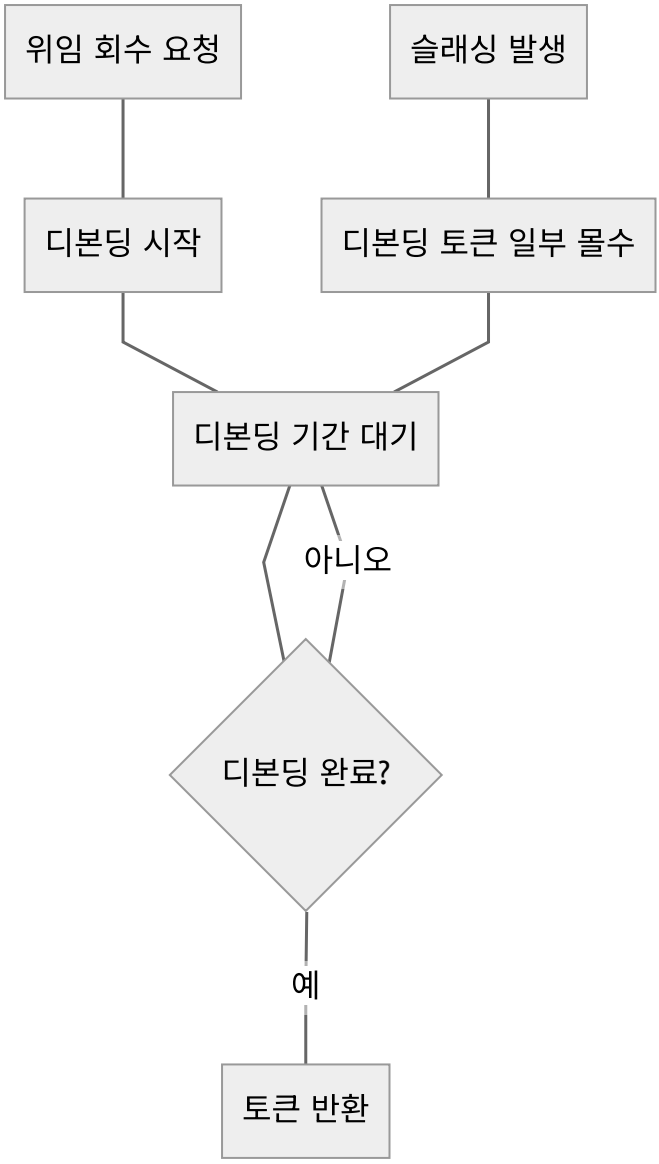
1. 위임 회수 요청

```
// 위임 회수 구조체
type ReclaimEscrow struct {
Account Address `json:"account"` // 밸리데이터 주소
Shares quantity.Quantity `json:"shares"` // 회수할 쉐어 양
}

// 위임 회수 처리
func (app *stakingApplication) reclaimEscrow(
ctx *api.Context,
```

```
state *stakingState.MutableState,
from Address,
reclaim *ReclaimEscrow,
) error {
// 디본딩 시작
debondingDelegation := &api.DebondingDelegation{
Shares: reclaim.Shares,
DebondEndTime: ctx.Now().Add(app.state.ConsensusParameters().DebondingInterval),
}
// 활성 쉐어에서 디본딩 쉐어로 이동
return  state.SetDebondingDelegation(from, reclaim.Account, debondingDelegation)
}
```

2. 디본딩 기간



3. 디본딩 완료 및 토큰 반환

```
// 디본딩 완료 처리
func (app *stakingApplication) processDebondingDelegations(
ctx *api.Context,
state *stakingState.MutableState,
epoch beacon.EpochTime,
) error {
// 만료된 디본딩 위임 조회
expiredDebonding := state.ExpiredDebondingDelegations(epoch)
```

```
for delegator, delegations := range expiredDebonding {
for validator, debonding := range delegations {
// 토큰을 일반 계정으로 반환
err := app.returnDebondedTokens(state, delegator, validator, debonding)
if err != nil {
return err
}
}
}
return nil
}
```

슬래싱 프로세스

슬래싱은 밸리데이터의 악의적 행동에 대한 자동 처벌 메커니즘입니다.

1. 슬래싱 조건

- **이중 서명**: 동일한 높이에서 서로 다른 블록에 서명
- **다운타임**: 장기간 블록 생성 또는 투표 미참여
- **부적절한 행동**: 프로토콜 위반 행위

2. 슬래싱 실행

```
// 슬래싱 이벤트 구조체
type SlashingEvent struct {
```

```
Validator Address `json:"validator"`

Reason SlashingReason `json:"reason"`

Amount quantity.Quantity `json:"amount"`

Epoch beacon.EpochTime `json:"epoch"`

}



// 슬래싱 처리

func (app *stakingApplication) slashValidator(

ctx *api.Context,

state *stakingState.MutableState,

validator Address,

reason SlashingReason,

percentage  uint64,

) error {

// 밸리데이터 계정 조회

account, err := state.Account(validator)

if  err != nil {

return  err

}

// 슬래싱할 양 계산

totalStake := account.Escrow.Active.Balance

slashAmount := totalStake.Clone()

slashAmount.Mul(quantity.NewFromUint64(percentage))
```

```
slashAmount.Quo(quantity.NewFromUint64(100))

// 활성 스테이크에서 슬래싱

activeSlashed := slashAmount.Clone()

if  activeSlashed.Cmp(&account.Escrow.Active.Balance) > 0 {

activeSlashed = account.Escrow.Active.Balance.Clone()

}

// 디본딩 스테이크에서도 슬래싱

debondingSlashed := quantity.NewQuantity()

remaining := slashAmount.Clone()

remaining.Sub(activeSlashed)

if !remaining.IsZero() {

debondingSlashed = app.slashDebondingDelegations(

state, validator, remaining,

)

}

// 슬래싱된 토큰을 공통 풀로 이동

commonPool, err := state.CommonPool()

if  err != nil {

return  err

}

totalSlashed := activeSlashed.Clone()

totalSlashed.Add(debondingSlashed)

commonPool.Add(totalSlashed)
```

```
// 슬래싱 이벤트 발생
ctx.EmitEvent(&api.TakeEscrowEvent{
Owner: validator,
Amount: *totalSlashed,
DebondingAmount: *debondingSlashed,
})
return  nil
}
```

보상 분배 메커니즘

보상 구조

Oasis Core의 보상 시스템은 다음과 같은 구성 요소로 이루어집니다:

1. **블록 보상**: 새로운 블록 생성에 대한 보상
2. **트랜잭션 수수료**: 트랜잭션 처리 수수료
3. **위임 보상**: 위임자에게 분배되는 보상
4. **밸리데이터 수수료**: 밸리데이터가 가져가는 수수료

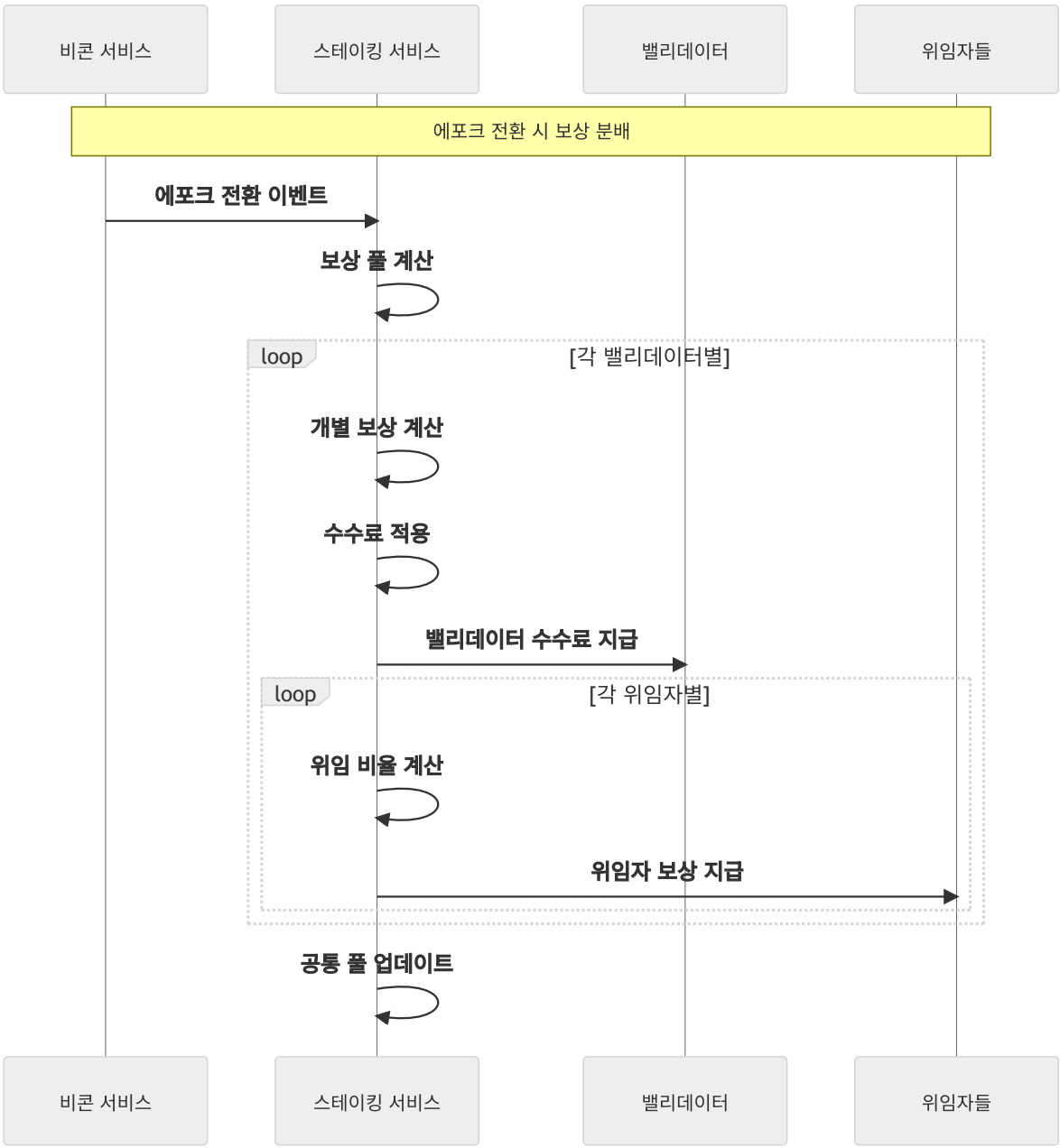
보상 계산 공식

```
// 보상 계산 구조체
type RewardCalculation struct {
TotalReward quantity.Quantity // 총 보상
ValidatorFee quantity.Quantity // 밸리데이터 수수료
```

```
DelegatorReward quantity.Quantity // 위임자 보상
CommissionRate quantity.Quantity // 수수료율
}


// 보상 분배 계산
func calculateRewardDistribution(
totalReward *quantity.Quantity,
commissionRate *quantity.Quantity,
) *RewardCalculation {
calc := &RewardCalculation{
TotalReward: *totalReward.Clone(),
CommissionRate: *commissionRate.Clone(),
}
// 밸리데이터 수수료 = 총 보상 × 수수료율
calc.ValidatorFee = calc.TotalReward.Clone()
calc.ValidatorFee.Mul(commissionRate)
calc.ValidatorFee.Quo(quantity.NewFromUint64(100))
// 위임자 보상 = 총 보상 - 밸리데이터 수수료
calc.DelegatorReward = calc.TotalReward.Clone()
calc.DelegatorReward.Sub(&calc.ValidatorFee)
return calc
}
```

에포크별 보상 분배



수수료 스케줄

밸리데이터는 시간에 따라 변화하는 수수료 스케줄을 설정할 수 있습니다.

```go
// 수수료 스케줄 구조체
type CommissionSchedule struct {
Rates []CommissionRateStep `json:"rates,omitempty"`
Bounds []CommissionRateBoundStep `json:"bounds,omitempty"`
}

type CommissionRateStep struct {
Start beacon.EpochTime `json:"start,omitempty"`
Rate quantity.Quantity `json:"rate,omitempty"`
}

// 수수료 스케줄 수정
type AmendCommissionSchedule struct {
Amendment CommissionSchedule `json:"amendment"`
}

// 현재 수수료율 계산
func (cs *CommissionSchedule) CurrentRate(epoch beacon.EpochTime) *quantity.Quan
var  currentRate *quantity.Quantity
for  _, step := range  cs.Rates {
if  epoch >= step.Start {
currentRate = &step.Rate
```

```go
} else {
break
}
}
if currentRate == nil {
return quantity.NewQuantity() // 0% 수수료
}
return currentRate.Clone()
}
```

API 및 인터페이스

스테이킹 백엔드 인터페이스

```go
// Backend는 스테이킹 구현을 위한 인터페이스입니다
type Backend interface {
// TokenSymbol은 토큰의 심볼을 반환합니다
TokenSymbol(ctx context.Context, height int64) (string, error)
// TokenValueExponent는 토큰의 소수점 자릿수를 반환합니다
TokenValueExponent(ctx context.Context, height int64) (uint8, error)
// TotalSupply는 총 토큰 공급량을 반환합니다
TotalSupply(ctx context.Context, height int64) (*quantity.Quantity, error)
// CommonPool은 공통 풀 잔액을 반환합니다
```

```
CommonPool(ctx context.Context, height  int64) (*quantity.Quantity, error)

// Account는 지정된 계정의 정보를 반환합니다

Account(ctx context.Context, query *OwnerQuery) (*Account, error)

// DelegationsFor는 지정된 소유자의 위임 목록을 반환합니다

DelegationsFor(ctx context.Context, query *OwnerQuery) (map[Address]*Delegation,

// DebondingDelegationsFor는 디본딩 중인 위임 목록을 반환합니다

DebondingDelegationsFor(ctx context.Context, query *OwnerQuery) (map[Address][]*

// Threshold는 특정 임계값을 반환합니다

Threshold(ctx context.Context, query *ThresholdQuery) (*quantity.Quantity, error

// ConsensusParameters는 스테이킹 합의 파라미터를 반환합니다

ConsensusParameters(ctx context.Context, height  int64) (*ConsensusParameters, e

// GetEvents는 지정된 높이의 이벤트들을 반환합니다

GetEvents(ctx context.Context, height  int64) ([]*Event, error)

// WatchEvents는 이벤트 스트림을 반환합니다

WatchEvents(ctx context.Context) (<-chan *Event, pubsub.ClosableSubscription, er

}
```

주요 데이터 구조

```
// Account는 스테이킹 계정 정보입니다

type Account struct {

General GeneralAccount `json:"general"`

Escrow EscrowAccount `json:"escrow"`
```

```go
}

// GeneralAccount는 일반 계정 정보입니다
type GeneralAccount struct {
Balance quantity.Quantity `json:"balance"`
Nonce  uint64  `json:"nonce"`
Allowances  map[Address]quantity.Quantity `json:"allowances,omitempty"`
}

// EscrowAccount는 에스크로 계정 정보입니다
type EscrowAccount struct {
Active SharePool `json:"active"`
Debonding SharePool `json:"debonding"`
CommissionSchedule CommissionSchedule `json:"commission_schedule"`
StakeAccumulator StakeAccumulator `json:"stake_accumulator"`
}

// Delegation은 위임 정보입니다
type Delegation struct {
Shares quantity.Quantity `json:"shares"`
}

// DebondingDelegation은 디본딩 위임 정보입니다
```

```
type DebondingDelegation struct {
Shares quantity.Quantity `json:"shares"`
DebondEndTime beacon.EpochTime `json:"debond_end_time"`
}
```

gRPC API

스테이킹 서비스 정의

```
service StakingService {
// GetAccount는 계정 정보를 조회합니다
rpc  GetAccount(GetAccountRequest) returns (GetAccountResponse);
// GetDelegations는 위임 정보를 조회합니다
rpc  GetDelegations(GetDelegationsRequest) returns (GetDelegationsResponse);
// GetDebondingDelegations는 디본딩 위임 정보를 조회합니다
rpc  GetDebondingDelegations(GetDebondingDelegationsRequest) returns (GetDebondi
// GetTotalSupply는 총 공급량을 조회합니다
rpc  GetTotalSupply(GetTotalSupplyRequest) returns (GetTotalSupplyResponse);
// GetCommonPool은 공통 풀을 조회합니다
rpc  GetCommonPool(GetCommonPoolRequest) returns (GetCommonPoolResponse);
// WatchEvents는 스테이킹 이벤트를 스트리밍합니다
rpc  WatchEvents(WatchEventsRequest) returns (stream StakingEvent);
}
```

메시지 타입

```
message GetAccountRequest {
int64  height = 1;
bytes  owner = 2;
}

message GetAccountResponse {
Account  account = 1;
}

message Account {
GeneralAccount  general = 1;
EscrowAccount  escrow = 2;
}

message StakingEvent {
int64  height = 1;
bytes  tx_hash = 2;
oneof  event {
TransferEvent  transfer = 3;
EscrowEvent  escrow = 4;
BurnEvent  burn = 5;
```

```
AllowanceChangeEvent  allowance_change = 6;

}

}
```

설정 및 구성

스테이킹 설정 구조

```yaml
staking:

# 토큰 설정

token_symbol: "ROSE"

token_value_exponent: 9

# 임계값 설정

thresholds:

entity: "100000000000"  # 엔티티 등록 임계값

node_validator: "100000000000"  # 밸리데이터 노드 임계값

node_compute: "50000000000"  # 컴퓨트 노드 임계값

runtime_compute: "50000000000"  # 컴퓨트 런타임 임계값

runtime_keymanager: "50000000000"  # 키매니저 런타임 임계값

# 위임 설정

min_delegation_amount: "10000000000"  # 최소 위임 금액

debonding_interval: 336  # 디본딩 기간 (에포크)

# 수수료 설정
```

```yaml
fee_split_weight_propose: "2"  # 블록 제안 수수료 가중치
fee_split_weight_vote: "1"  # 투표 수수료 가중치
fee_split_weight_next_propose: "1"  # 다음 제안자 수수료 가중치
# 보상 설정
reward_schedule:
- until: 3696
scale: "2595"
- until: 7392
scale: "2080"
# 슬래싱 설정
slashing:
double_sign_slash_percent: 5  # 이중 서명 슬래싱 비율 (%)
downtime_slash_percent: 1  # 다운타임 슬래싱 비율 (%)
signed_blocks_window: 10000  # 서명 블록 윈도우
min_signed_per_window: 5000  # 윈도우당 최소 서명 수
```

주요 설정 옵션

옵션 이름	타입	기본값	설명
`token_symbol`	string	“ROSE”	토큰 심볼
`token_value_exponent`	uint8	9	토큰 소수점 자릿수
`min_delegation_amount`	string	“10000000000”	최소 위임 금액
`debonding_interval`	uint64	336	디본딩 기간 (에포크)

| `thresholds.entity` | string | “100000000000” | 엔티티 등록 임계값 |

| `thresholds.node_validator` | string | “100000000000” | 밸리데이터 임계값 |

환경 변수

| 환경 변수 | 설명 | 예시 |

|----------|------|------|

| `OASIS_STAKING_TOKEN_SYMBOL` | 토큰 심볼 | `ROSE` |

| `OASIS_STAKING_MIN_DELEGATION` | 최소 위임 금액 | `10000000000` |

| `OASIS_STAKING_DEBONDING_INTERVAL` | 디본딩 기간 | `336` |

코드 예제

기본 사용법

```go
package main


import (
"context"
"fmt"
"log"
"github.com/oasisprotocol/oasis-core/go/common/quantity"
"github.com/oasisprotocol/oasis-core/go/consensus/api"
"github.com/oasisprotocol/oasis-core/go/staking/api"
)
```

```go
func main() {
// 합의 서비스 연결
consensus, err := api.NewConsensusClient("unix:/tmp/oasis-node-consensus.sock")
if err != nil {
log.Fatal("합의 클라이언트 생성 실패:", err)
}
// 스테이킹 백엔드 가져오기
staking := consensus.Staking()
ctx := context.Background()
// 총 공급량 조회
totalSupply, err := staking.TotalSupply(ctx, api.HeightLatest)
if err != nil {
log.Fatal("총 공급량 조회 실패:", err)
}
fmt.Printf("총 공급량: %s\n", totalSupply.String())
// 공통 풀 조회
commonPool, err := staking.CommonPool(ctx, api.HeightLatest)
if err != nil {
log.Fatal("공통 풀 조회 실패:", err)
}
fmt.Printf("공통 풀: %s\n", commonPool.String())
```

```
}
```

계정 정보 조회

```
// 계정 정보 조회
func getAccountInfo(staking api.Backend, address api.Address) (*api.Account, er
ctx := context.Background()
query := &api.OwnerQuery{
Height: api.HeightLatest,
Owner: address,
}
account, err := staking.Account(ctx, query)
if err != nil {
return nil, fmt.Errorf("계정 조회 실패: %w", err)
}
fmt.Printf("계정 주소: %s\n", address)
fmt.Printf("일반 잔액: %s\n", account.General.Balance.String())
fmt.Printf("에스크로 활성: %s\n", account.Escrow.Active.Balance.String())
fmt.Printf("에스크로 디본딩: %s\n", account.Escrow.Debonding.Balance.String())
return account, nil
}
```

위임 정보 조회

```go
// 위임 정보 조회
func getDelegationInfo(staking api.Backend, delegator api.Address) error {
ctx := context.Background()
query := &api.OwnerQuery{
Height: api.HeightLatest,
Owner: delegator,
}
// 위임 정보 조회
delegations, err := staking.DelegationsFor(ctx, query)
if err != nil {
return fmt.Errorf("위임 정보 조회 실패: %w", err)
}
fmt.Printf("위임자: %s\n", delegator)
for validator, delegation := range delegations {
fmt.Printf(" 밸리데이터: %s, 쉐어: %s\n",
validator, delegation.Shares.String())
}
// 디본딩 위임 정보 조회
debondingDelegations, err := staking.DebondingDelegationsFor(ctx, query)
if err != nil {
return fmt.Errorf("디본딩 위임 조회 실패: %w", err)
}
```

```go
fmt.Println("디본딩 위임:")
for validator, debondings := range debondingDelegations {
for _, debonding := range debondings {
fmt.Printf(" 밸리데이터: %s, 쉐어: %s, 완료 시간: %d\n",
validator, debonding.Shares.String(), debonding.DebondEndTime)
}
}
return nil
}
```

위임 트랜잭션 생성

```go
// 위임 트랜잭션 생성
func createDelegationTx(
validator api.Address,
amount *quantity.Quantity,
nonce uint64,
) *transaction.Transaction {
escrow := &api.Escrow{
Account: validator,
Amount: *amount,
}
fee := &transaction.Fee{
```

```
Amount: *quantity.NewFromUint64(1000),
Gas: 1000,
}
return  api.NewAddEscrowTx(nonce, fee, escrow)
}


// 위임 회수 트랜잭션 생성
func  createReclaimTx(
validator api.Address,
shares *quantity.Quantity,
nonce  uint64,
) *transaction.Transaction {
reclaim := &api.ReclaimEscrow{
Account: validator,
Shares: *shares,
}
fee := &transaction.Fee{
Amount: *quantity.NewFromUint64(1000),
Gas: 1000,
}
return  api.NewReclaimEscrowTx(nonce, fee, reclaim)
}
```

이벤트 모니터링

```go
// 스테이킹 이벤트 모니터링
func watchStakingEvents(staking api.Backend) error {
ctx := context.Background()
// 이벤트 스트림 구독
eventCh, sub, err := staking.WatchEvents(ctx)
if err != nil {
return fmt.Errorf("이벤트 구독 실패: %w", err)
}
defer sub.Close()
fmt.Println("스테이킹 이벤트를 모니터링합니다...")
for {
select {
case event := <-eventCh:
handleStakingEvent(event)
case <-ctx.Done():
return ctx.Err()
}
}
}

func handleStakingEvent(event *api.Event) {
```

```
fmt.Printf("블록 높이: %d\n", event.Height)
if  event.Transfer != nil {
fmt.Printf("전송: %s -> %s, 금액: %s\n",
event.Transfer.From,
event.Transfer.To,
event.Transfer.Amount.String())
}
if  event.Escrow != nil {
if  event.Escrow.Add != nil {
fmt.Printf("위임 추가: %s -> %s, 금액: %s\n",
event.Escrow.Add.Owner,
event.Escrow.Add.Escrow,
event.Escrow.Add.Amount.String())
}
if  event.Escrow.Reclaim != nil {
fmt.Printf("위임 회수: %s <- %s, 금액: %s\n",
event.Escrow.Reclaim.Owner,
event.Escrow.Reclaim.Escrow,
event.Escrow.Reclaim.Amount.String())
}
}
if  event.Burn != nil {
fmt.Printf("토큰 소각: %s, 금액: %s\n",
```

```
event.Burn.Owner,
event.Burn.Amount.String())
}
}
```

CLI 사용 예제

```
# 계정 정보 조회
oasis-node staking account --address oasis1...

# 위임 정보 조회
oasis-node staking delegations --address oasis1...

# 디본딩 위임 조회
oasis-node staking debonding-delegations --address oasis1...

# 총 공급량 조회
oasis-node staking total-supply

# 공통 풀 조회
oasis-node staking common-pool

# 임계값 조회
```

```
oasis-node staking threshold --kind node-validator


# 위임 트랜잭션 생성
oasis-node staking gen-escrow \
--account.dir /path/to/account \
--transaction.file escrow.json \
--escrow.account oasis1... \
--escrow.amount 1000000000


# 위임 회수 트랜잭션 생성
oasis-node staking gen-reclaim-escrow \
--account.dir /path/to/account \
--transaction.file reclaim.json \
--escrow.account oasis1... \
--escrow.shares 1000000000
```

모니터링 및 메트릭

주요 메트릭

메트릭 이름	타입	설명
`oasis_staking_total_supply`	Gauge	총 토큰 공급량
`oasis_staking_common_pool`	Gauge	공통 풀 잔액

| `oasis_staking_delegations_total` | Counter | 총 위임 수 |

| `oasis_staking_debonding_delegations_total` | Counter | 총 디본딩 위임 수 |

| `oasis_staking_rewards_distributed` | Counter | 분배된 보상 총액 |

| `oasis_staking_slashing_events_total` | Counter | 슬래싱 이벤트 수 |

| `oasis_staking_active_validators` | Gauge | 활성 밸리데이터 수 |

| `oasis_staking_total_voting_power` | Gauge | 총 투표권 |

헬스 체크

```
# 스테이킹 서비스 상태 확인
curl http://localhost:9001/staking/status

# 응답 예시
{
"total_supply": "1000000000000000000",
"common_pool": "100000000000000000",
"active_validators": 100,
"total_delegations": 50000,
"debonding_delegations": 1000,
"last_reward_distribution": 12345
}
```

로그 모니터링

```
# 스테이킹 관련 로그 확인
journalctl -u oasis-node -f | grep staking

# 위임 관련 로그 확인
journalctl -u oasis-node | grep "delegation\|escrow"

# 슬래싱 관련 로그 확인
journalctl -u oasis-node | grep "slash"

# 보상 분배 로그 확인
journalctl -u oasis-node | grep "reward"
```

문제 해결

일반적인 문제

문제 1: 위임이 처리되지 않음

증상: 위임 트랜잭션이 실패하거나 반영되지 않음

원인:

- 잔액 부족
- 최소 위임 금액 미달
- 밸리데이터 주소 오류
- 가스 부족

해결 방법:

1. 계정 잔액 확인

```
oasis-node staking account --address oasis1...
```

2. 최소 위임 금액 확인

```
oasis-node staking show-parameters | grep  min_delegation
```

3. 밸리데이터 주소 검증

```
oasis-node registry node list | grep  oasis1...
```

문제 2: 디본딩이 완료되지 않음

증상: 디본딩 기간이 지났는데 토큰이 반환되지 않음

원인:

- 디본딩 기간 계산 오류
- 슬래싱으로 인한 토큰 감소
- 네트워크 동기화 문제

해결 방법:

1. 디본딩 상태 확인

```
oasis-node staking debonding-delegations --address oasis1...
```

2. 현재 에포크 확인

```
oasis-node beacon show-epoch
```

3. 슬래싱 이벤트 확인

```
oasis-node staking events --height-range 12000:13000
```

문제 3: 보상이 분배되지 않음

증상: 위임했는데 보상을 받지 못함

원인:

- 밸리데이터가 비활성 상태
- 수수료율이 100%
- 위임 시점 문제

해결 방법:

1. 밸리데이터 상태 확인

```
oasis-node registry node list --verbose | grep  oasis1...
```

2. 수수료 스케줄 확인

```
oasis-node staking account --address oasis1... | grep  commission
```

3. 보상 분배 이벤트 확인

로그 분석

```
# 위임 관련 에러 로그
```

```
grep "escrow.*ERROR" /var/log/oasis/node.log

# 슬래싱 이벤트 로그
grep "slash\|take_escrow" /var/log/oasis/node.log

# 보상 분배 로그
grep "reward\|commission" /var/log/oasis/node.log

# 디본딩 처리 로그
grep "debonding\|reclaim" /var/log/oasis/node.log
```

성능 튜닝

위임 처리 최적화

```
staking:
# 배치 처리 크기 조정
max_delegations_per_batch: 1000
# 캐시 설정
delegation_cache_size: 10000
```

보상 계산 최적화

```
staking:
```

```yaml
# 보상 계산 간격
reward_calculation_interval: 1
# 수수료 캐시 설정
commission_cache_ttl: 3600
```

참조

관련 문서

- 합의 프로세스
- 비콘 서비스
- 스케줄러 서비스
- 거버넌스 서비스
- 아키텍처 개요

소스 코드

- 스테이킹 API
- 스테이킹 구현
- 스테이킹 애플리케이션

외부 리소스

- Proof of Stake 개념
- 위임 증명 (DPoS)
- 슬래싱 메커니즘

마지막 업데이트: 2024년 1월

작성자: Oasis Core 문서화 팀

검토자: 스테이킹 개발팀